

cjquant 软件架构与使用说明

版本：0.1.0

定位：面向国内资管机构（公募 FOF、理财子、券商资管、家族办公室等）的 **场外基金量化投研 / 资产配置 / 组合管理** 框架。

本文档系统梳理 cjquant 的整体软件架构、核心模块职责、目录组织与典型使用方式，便于开发者快速理解、二次开发与生产部署。

1. 项目总览

cjquant 不同于面向场内高频/日频撮合交易的 [vn.py](#) / [qlib](#) / [backtrader](#)，它围绕场外资产的“份额—净值—流动性时间轴”模型构建，重点解决：

- **场外申赎流程仿真**：T+n 确认 / T+n 到账、阶梯赎回费、冲击成本、FIFO 批次计费
- **异构低频数据治理**：公募日频、私募周/月频、宏观季频的对齐与 Proxy 映射插值
- **场外资产穿透归因**：RBSA（基于净值的风格分析）与 HBA（基于持仓的归因）
- **场外资产组合优化**：风险平价、HRP、均值方差、Black-Litterman，并自动做协方差收缩
- **下单文件落地**：直接对接恒生 O32 / 微信理财通格式
- **可视化投研终端**：基于 FastAPI + PyWebView 的桌面级 GUI

工程由两条主线组成：

1. **cjquant/** —— **Python 量化库**（可作为 `pip install cjquant` 引入的纯库）
2. **app/** —— **桌面投研终端**（基于 FastAPI 提供 REST API，PyWebView 包装为桌面应用）

2. 目录结构

```

cjquant/
├─ cjquant/
│   │   └─ __init__.py
│   │   └─ data/
│   │       │   └─ pipeline.py
│   │       │   └─ aligner.py
│   │       │   └─ schema.py
│   │       │   └─ exceptions.py
│   │       └─ providers/
│   │           │   └─ base.py
│   │           │   └─ public.py
│   │           └─ private.py
│   └─ backtest/
│       │   └─ engine.py
│       │   └─ portfolio.py
│       │   └─ model.py
│       │   └─ fee.py
│       │   └─ slippage.py
│       │   └─ exceptions.py
│   └─ look_through/
│       │   └─ engine.py
│       │   └─ rbsa_engine.py
│       │   └─ hba_engine.py
│       │   └─ model.py
│   └─ optimizer/
│       │   └─ base.py
│       │   └─ traditional.py
│       │   └─ machine_learning.py
│       │   └─ black_litterman.py
│       │   └─ utils.py
│   └─ visualizer/
│       │   └─ reporter.py
│       │   └─ plotter.py
│       │   └─ metrics.py
│   └─ execution/
│       │   └─ api.py
│       │   └─ o32.py
│
└─ app/
    └─ gui.py

# 核心 Python 库
# 顶层导出: data_api / 引擎 / 优化器 / 分析器 / 执行器
# 数据接入、对齐与治理
# DataPipeline & data_api 单例
# 异构频率对齐 (ffill / linear / proxy_mapping)
# 标准化 NAV 数据 schema 校验

# BaseProvider 抽象
# 公募基金 (akshare / 东财)
# 私募基金本地文件 (csv/xlsx)
# 场外回测引擎
# OTCBacktestEngine 主引擎
# TransitCashAccount / FundPosition (FIFO 批次)
# Order / TradeRecord / PositionLot
# OTCFeeModel 阶梯赎回费
# 平方根冲击成本 / 自定义 / 零冲击

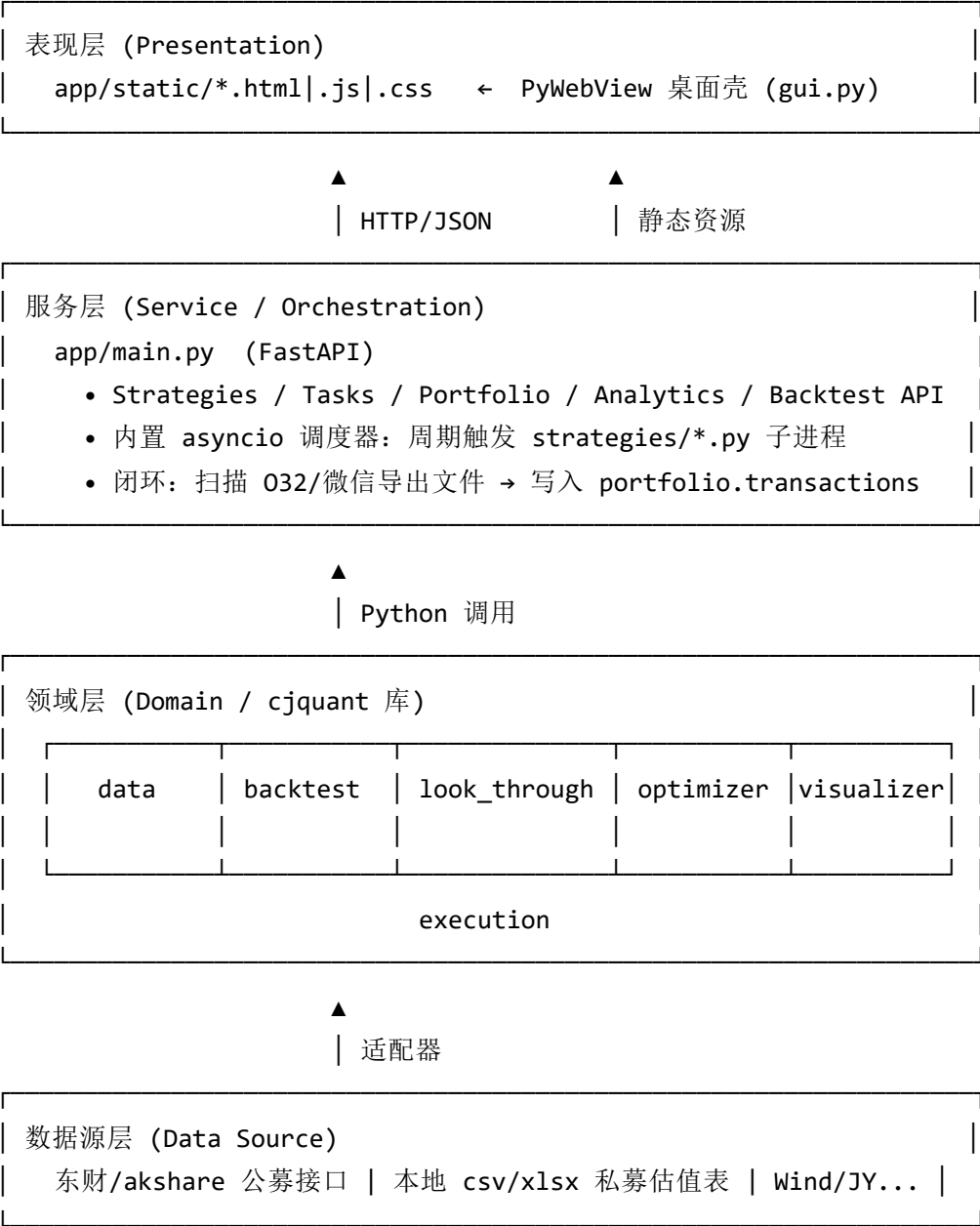
# 穿透分析 (黑盒拆解)
# LookThroughAnalyzer 统一入口
# Returns-Based Style Analysis
# Holdings-Based Analysis
# ExposureResult
# 组合优化
# BaseOptimizer
# RiskParity / MeanVariance
# HRP (层次风险平价)
# Black-Litterman
# Ledoit-Wolf 协方差收缩
# 报告生成
# CJQuantReporter (HTML/PNG/PDF)
# Plotly 交互图 + Matplotlib 静态汇总
# 累计/年化/夏普/索提诺/卡玛/最大回撤
# 真实下单导出
# OTCExecutor (极简 buy/sell)
# O32OrderExporter (恒生 O32 csv/xlsx)

# 投研桌面终端
# PyWebView 入口 (自动拉起 uvicorn)
```

```
|   ├── main.py           # FastAPI 应用 + 后台调度器
|   ├── tasks.json        # 调度任务清单（持久化）
|   ├── portfolio.json    # 账户组合状态（持久化）
|   ├── strategies/       # 用户策略目录（可在 GUI 内编辑）
|   ├── logs/             # 策略运行日志
|   └── static/           # 前端单页 (index.html / app.js / style.css)
|
|── run_gui.py            # 一键启动桌面终端
|── pyproject.toml        # 包元数据与依赖
|── README.md             # 产品介绍与代码示例
└── doc/                  # （本文档所在目录）
```

3. 软件架构

3.1 分层视图



3.2 运行时拓扑（桌面终端）

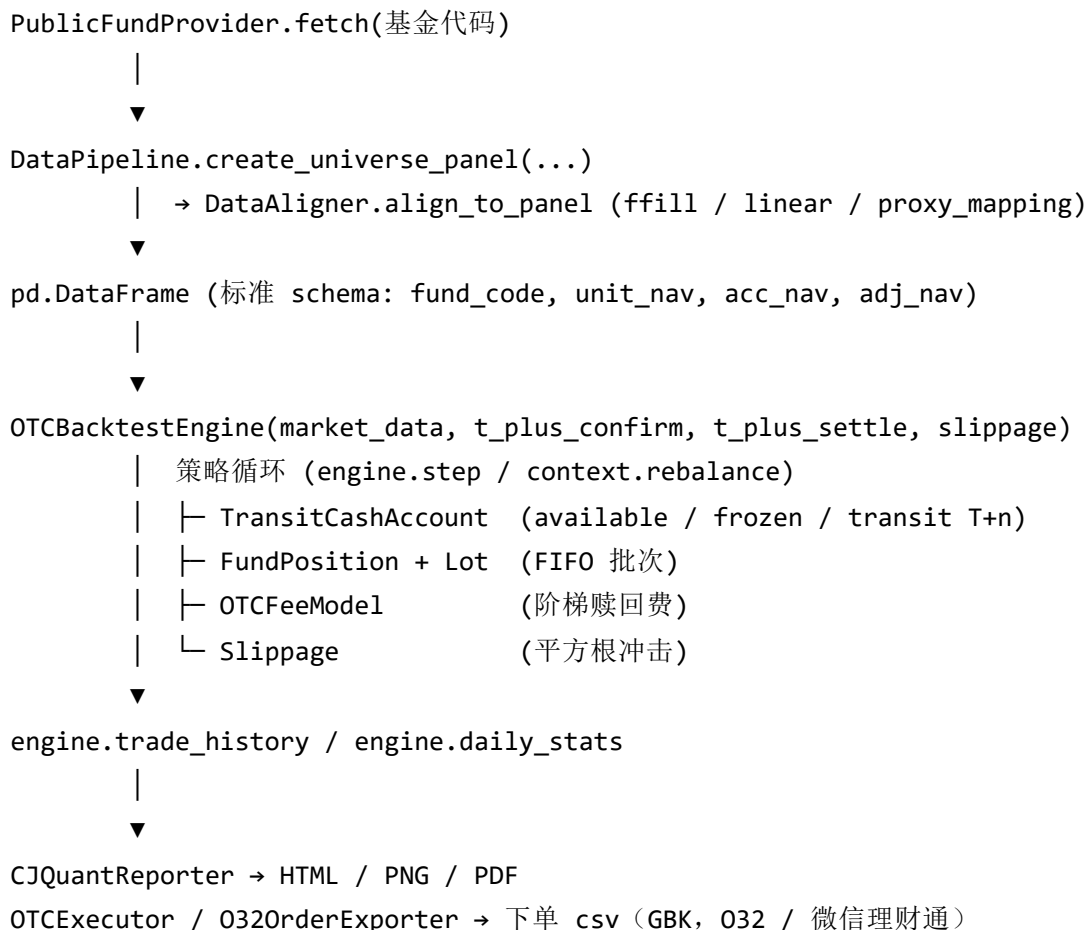
run_gui.py → app/gui.py :

1. 检查 127.0.0.1:8000 是否有 FastAPI 实例;
2. 若无则以子进程方式启动 uvicorn app.main:app ;
3. 通过 pywebview 创建窗口, 加载 http://127.0.0.1:8000 ;

4. 窗口关闭时自动 terminate 子进程。

app/main.py 启动时会注册 scheduler_loop (每 5s 轮询一次 tasks.json)，按 interval 周期把启用的策略以独立 Python 子进程拉起，输出实时重定向到 app/logs/<task>.log。

3.3 数据流（典型回测）



4. 核心模块详解

4.1 cjquant.data — 数据接入与治理

- **PublicFundProvider**：基于 akshare.fund_open_fund_info_em 拉取公募基金的单位净值与累计净值，合并为统一 schema (unit_nav / acc_nav / adj_nav)，含重试机制。
- **PrivateFundLocalProvider**：解析私募排排网/托管行导出的 csv/xlsx，自动用 chardet 探测编码、处理千分位，缺少 adj_nav 时基于单位净值收益率近似重建（私募分红极少）。
- **DataAligner**：将多基金 NAV 对齐到统一时间轴，提供三种插值策略：

- ffill —— 前向填充
- linear —— 线性插值
- proxy_mapping —— 引入一只指数基金作为 proxy，按对数收益率分摊残差 (Alpha)，适合低频私募/理财补全日频估算
- FundDataSchema.validate：标准列校验、tz 去除、重复日期保留最后、按日期排序。
- 顶层 data_api = DataPipeline()：导入即用的单例。

4.2 cjquant.backtest — 场外回测引擎

核心类是 OTCBacktestEngine，关键设计点：

概念	说明
TransitCashAccount	三段式现金：available_cash / frozen_cash（提交 BUY 即冻结） / transit_queue（SELL 到账队列，按结算日 pop）
FundPosition + PositionLot	按确认日的 FIFO 批次持仓，赎回时按批次计算"持有自然日"匹配阶梯费率
OTCFeeModel	默认阶梯：<7d 1.5%、<30d 0.75%、<180d 0.5%、≥180d 免费；可 set_sell_tiers 自定义
Slippage	SquareRootSlippage (Impact = $\lambda \cdot \text{NAV} \cdot \sqrt{(V/AUM)}$)、CustomLambdaSlippage、ZeroSlippage
t_plus_confirm / t_plus_settle	净值确认 T+n / 赎回资金到账 T+n
_nav_cache	引擎初始化时按交易日历对每只基金 ffill，避免节假日缺净值导致市值瞬时归零
daily_stats	每日记录 available / frozen / transit / market_value / total_assets，便于绘图与归因

提交订单与撮合：

```
engine.submit_order(fund_code, 'BUY', value=...) # 立即冻结现金
engine.submit_order(fund_code, 'SELL', shares=...) # 立即冻结份额
engine.step() # 推进 1 个交易日：到账 → 撮合确认 → 记录每日结
```

4.3 cjquant.look_through — 穿透分析

统一入口 `LookThroughAnalyzer(method='RBSA' | 'HBA', ...)`：

- **RBSA** (`rbbsa_engine.py`)：Sharpe 法。带约束 SLSQP ($\sum w=1$, $w \geq 0$) 的非负回归，输出每个因子的暴露权重和 R^2 ；可指定 `rolling_window` 做滚动风格分析。
- **HBA** (`hba_engine.py`)：基于 `holdings_df` (`fund_code` / `stock_code` / `weight` / `industry` / `style`) 逐基金按行业或风格归集，再以组合权重加权穿透。

`app/main.py` 中 `/api/analytics/look_through` 演示了将 HBA 与"动态生成 mock 持仓"结合用于 GUI 的展示。

4.4 cjquant.optimizer — 组合优化

- `BaseOptimizer`：统一 `_validate_weights` 做归一化。
- `RiskParityOptimizer`：SLSQP 最小化"风险贡献离散度"。
- `MeanVarianceOptimizer`：最大化夏普（最小化负夏普）。
- `HRPOptimizer`：相关系数 $\rightarrow$ 距离矩阵 $\rightarrow$ 单联动聚类 $\rightarrow$ 准对角化 $\rightarrow$ 递归二分配权。
- `BlackLittermanOptimizer`：Idzorek 简化法默认构造 Ω ，融合先验 Π 与观点 (P, Q) 。
- `utils.calculate_shrunk_covariance`：`sklearn.covariance.LedoitWolf` 协方差收缩，解决场外低频小样本下的奇异性问题。

4.5 cjquant.visualizer — 报告生成

- `QualityMetrics`：累计/年化收益、年化波动、夏普、索提诺、卡玛、最大回撤、平均现金占比、运行天数。
- `ReportPlotter.create_interactive_figure`：Plotly 4 子图（净值 / 回撤 / 资产构成堆叠 / 日收益柱）。
- `ReportPlotter.create_static_summary`：Matplotlib 月度收益热力图 + 净值+回撤双轴 + 月度换手。
- `CJQuantReporter`：消费 `engine.export_results_csv` 与 `engine.export_trades_csv` 的产物，导出 `html` / `png` / `pdf`（需要 `img2pdf`）。

4.6 cjquant.execution — 下单文件落地

- `OTCExecutor`：面向策略侧的极简 API。
 - `executor.buy(code, amount=...)` / `executor.sell(code, shares=...)`
 - `executor.execute(channels=["o32", "wechat"], file_prefix="...")` 同时输出多通道
- `O32OrderExporter`：恒生 O32 标准列（证券代码 / 业务类别 / 委托金额 / 委托数量 / 组合编号 / 资产单元 / 交易日期），默认 GBK 编码以兼容中文 Windows。

- 微信理财通通道则使用 UTF-8-SIG 编码，列名为"基金代码/交易类型/发生金额/发生份额/..."。

4.7 app/ — 投研桌面终端

REST API 概览（FastAPI on :8000）

分类	方法 + 路径	作用
策略文件	GET /api/strategies	列出 app/strategies/*.py
	GET /api/strategies/{name}	读取脚本内容
	PUT /api/strategies/{name}	覆写脚本内容
	POST /api/strategies/create	新建脚本
任务调度	GET /api/tasks	列出任务
	POST /api/tasks	新建定时任务（interval 秒）
	DELETE /api/tasks/{name}	删除任务
	POST /api/tasks/{name}/toggle	启用/暂停
	POST /api/tasks/{name}/run	立即触发一次（后台执行）
	GET /api/tasks/{name}/logs	拉取最近 300 行日志
账户组合	GET /api/portfolio	拉取组合（自动同步导出文件）
	POST /api/portfolio/update	覆写初始资金 / 现金 / 持仓
	POST /api/portfolio/refresh_navs	拉取最新净值并重估市值
分析	POST /api/analytics/optimize	调 RiskParityOptimizer / MeanVarianceOptimizer / HRP Optimizer
	POST /api/analytics/correlation	收益率相关性矩阵
	POST /api/analytics/performance	年化收益 / 波动 / 夏普 / 最大回撤
	POST /api/analytics/look_through	调 LookThroughAnalyzer (HBA)
	POST /api/analytics/backtest	调用 OTCBacktestEngine 跑回测，支持加载用户策略脚本
前端	GET /	返回 static/index.html

分类	方法 + 路径	作用
	/static/*	静态资源

调度与闭环

- scheduler_loop()：5s 轮询一次 tasks.json，对 enabled=True 且 next_run 已到期的任务，通过 asyncio.create_subprocess_exec 拉起独立解释器进程执行策略脚本，stdout/stderr 实时写入 logs/<task>.log。
- sync_exported_orders_to_portfolio()：每次访问 /api/portfolio 都会扫描根目录下的 *_o32.csv / *_wechat.csv，把订单条目记入 transactions，并把源文件重命名为 .processed_YYYYMMDD_HHMMSS 防重入。

策略脚本规范

两种形态：

1. **独立可执行脚本**（如 risk_parity_rebalance.py）：定义 run_strategy() 并在 __main__ 调用，调度器以子进程方式直接 python xxx.py 运行。
2. **回测 handle_bar 风格**（如 backtest_demo.py）：定义两个函数被 /api/analytics/backtest 通过 importlib 动态加载：

```
def init(context):          # 可选: context.funds / context.last_month = None
    ...

def handle_bar(context): # 必选: 每个交易日被调用
    context.current_date          # datetime
    context.current_date_idx      # int
    context.cash                  # 可用现金
    context.positions             # {code: total_shares}
    context.get_history_navs(code, N) # 取最近 N 个交易日 adj_nav 序列
    context.buy(code, value)
    context.sell(code, shares)
    context.rebalance({code: weight}) # 自动卖出旧仓 + 待 transit 清空后买入新仓
```

5. 安装与启动

5.1 环境要求

- Python ≥ 3.8
- 主要运行依赖：
pandas / numpy / scipy / scikit-learn / plotly / matplotlib / seaborn / akshare / chardet
- 桌面 GUI 额外依赖：fastapi / uvicorn / pywebview（首次启动 GUI 前需自行安装）

5.2 作为 Python 库使用

```
pip install cjquant
```

或在源码根目录开发安装：

```
pip install -e .
```

5.3 启动桌面投研终端

方式一：一键脚本（推荐）

```
python run_gui.py
```

方式二：手动分离启动 API 与前端

```
uvicorn app.main:app --host 127.0.0.1 --port 8000
```

浏览器访问 <http://127.0.0.1:8000>

启动后会自动：

- 在 app/strategies/ 加载现有策略
- 在 app/tasks.json 加载任务清单并启动周期调度器
- 在 app/portfolio.json 加载组合快照

6. 典型使用场景

6.1 跑一次完整的 FOF 回测

```
import pandas as pd
from cjquant.data.pipeline import DataPipeline
from cjquant.backtest.engine import OTCTestEngine
from cjquant.backtest.fee import OTCTestFeeModel
from cjquant.backtest.slippage import SquareRootSlippage

pipe = DataPipeline()
mkt = pd.concat([
    pipe.get_public_fund("000001.OF", "2023-01-01", "2023-12-31"),
    pipe.get_public_fund("000002.OF", "2023-01-01", "2023-12-31"),
])

fee = OTCTestFeeModel(buy_fee_rate=0.0015)
fee.set_sell_tiers([(7, 0.015), (30, 0.0075), (365, 0.005), (float("inf"), 0.0)])

engine = OTCTestEngine(
    market_data=mkt,
    initial_cash=1e7,
    t_plus_confirm=1,
    t_plus_settle=3,
    slippage_model=SquareRootSlippage(0.05, 1e8),
)
engine.fee_model = fee

while engine.current_date_idx < len(engine.trading_dates):
    if engine.current_date_idx == 10:
        engine.submit_order("000001.OF", "BUY", value=5_000_000)
        engine.submit_order("000002.OF", "BUY", value=4_000_000)
    engine.step()

engine.export_trades_csv("trades.csv")
engine.export_results_csv("results.csv")
```

6.2 风格穿透 (RBSA)

```
from cjquant.look_through.engine import LookThroughAnalyzer
import pandas as pd

factor_returns = pd.DataFrame({"HS300": hs300_nav.pct_change()}).dropna()
ana = LookThroughAnalyzer(method="RBSA", factor_returns=factor_returns)
res = ana.run(fund_nav.pct_change().dropna(), window=60)
print(res.exposures, res.r_squared)
```

6.3 HRP 组合优化 + HTML 报告

```
from cjquant.optimizer.machine_learning import HROptimizer
from cjquant.visualizer.reporter import CJQuantReporter

w = HROptimizer(returns_df).optimize()
print(w)

CJQuantReporter("results.csv", "trades.csv").generate_html("report.html")
```

6.4 下单文件导出

```
from cjquant.execution import OTCExecutor

exe = OTCExecutor(portfolio_id="FOF001", asset_unit="FOF001_01", output_dir="./")
exe.buy("000001.OF", amount=100_000)
exe.sell("000002.OF", shares=50_000)
exe.execute(channels=["o32", "wechat"], file_prefix="otc_orders")
# 产物: ./otc_orders_o32.csv (GBK) / ./otc_orders_wechat.csv (UTF-8-SIG)
```

6.5 桌面终端中配置定时调仓

1. 启动 `python run_gui.py`
2. 在"策略管理"页面新建/编辑 `my_strategy.py`，内部用 `cjquant.optimizer` 计算权重，再用 `OTCExecutor` 输出下单文件
3. 在"任务调度"页面创建任务 → 指向该策略 → 设置 interval 秒数 → 启用
4. 调度器到点会拉起子进程，日志显示在"任务日志"标签页
5. 输出的 `*_o32.csv` / `*_wechat.csv` 被根目录扫描后自动并入"账户组合 → 交易流水"

7. 扩展与二次开发

扩展点	做法
接入新数据源 (Wind/JYDB/ClickHouse)	继承 <code>data.providers.base.BaseProvider</code> ，实现 <code>fetch</code> 返回标准 schema；在 <code>DataPipeline</code> 内注册即可被上层无感切换
自定义费率/冲击	直接构造 <code>OTCFeeModel(...).set_sell_tiers(...)</code> ；或继承 <code>BaseSlippage</code> / 用 <code>CustomLambdaSlippage(func)</code>
自定义优化器	继承 <code>optimizer.base.BaseOptimizer</code> ，实现 <code>optimize()</code> 返回 <code>pd.Series</code> ；用 <code>_validate_weights</code> 归一化
自定义因子库	自行准备 <code>factor_returns DataFrame</code> 传入 <code>LookThroughAnalyzer(method='RBSA', factor_returns=...)</code>
新增 GUI API	在 <code>app/main.py</code> 新增 FastAPI 路由；如需异步任务用 <code>BackgroundTasks</code> 或 <code>asyncio.create_task</code>
新增下单通道	仿照 <code>execution/o32.py</code> 增加新的 <code>Exporter</code> ，并在 <code>OTCExecutor.execute</code> 中分支处理

8. 重要约定与注意事项

- **基金代码格式**：库内统一使用 `XXXXXX.OF`（如 `000001.OF`）；导出 O32/微信时通过 `strip_suffix=True` 去掉 `.OF` 适配券商/平台。
- **复权口径**：`adj_nav` 直接使用累计净值（已包含历史分红），不再用日增长率连乘重建，避免接口数据空值与字段不一致带来的曲线跳跃。
- **现金账户三段式**：策略可见的"现金"是 `available_cash`，但组合总值还需加上 `frozen_cash`（待买入冻结）和 `transit_queue`（赎回在途），引擎已在 `daily_stats` 中自动汇总。
- **赎回费按自然日**：阶梯比较使用 `confirm_date - lot.confirm_date` 的 `days`（自然日），符合公募实际计费规则。
- **GUI 端口默认 8000**：被占用时自行修改 `app/gui.py` 中的 `port` 与 `gui.py` 创建 `uvicorn` 的参数。
- **闭环交易流水的副作用**：每次访问 `/api/portfolio` 都会扫描并"消费"根目录的 `*_o32.csv`、`*_wechat.csv`，将其重命名为 `.processed_*`。如需重复导入，请改名后再放回。

9. Roadmap（节选自 README）

- 衍生品 / 雪球结构 / CTA 等另类资产抽象层
- 基于因子表现与组合持仓的 Brinson 归因自动化报告
- 合规风控引擎（事前拦截 + 机构白名单）

10. 协议

开源版本遵循 **GPLv3**；资管机构私有化定制 / 高级算子 / 系统级集成请参考商业授权方案。